

Final Project Report

Scalable Ride Matching System for a Driverless Taxi Fleet

Duong Bao Huy Tran | Student ID s224217463 | SIT314 | 1 October 2026

Project outcome

I designed, implemented and deployed an event-driven IoT prototype for a driverless taxi fleet. The system accepts vehicle telemetry over MQTT/TLS, stores live and historical positions, matches a rider to an available vehicle, sends a dispatch command, records trip completion and exposes authenticated REST endpoints. The complete ride loop and the telemetry pipeline were verified in AWS and MongoDB Atlas.

- Four application services ran on Amazon ECS Fargate: API, telemetry ingestion, matching and trip processing.
- The public Application Load Balancer health endpoint returned HTTP 200, while protected API routes rejected requests without a valid token.
- The final automated test run passed 15 of 15 tests. The largest HTTP stage completed 3,000 requests with 100 percent success at 360.33 requests per second.
- The telemetry experiment published 350 messages and verified 350 corresponding history documents in Atlas, with no remaining visible or in-flight SQS messages.

Implemented architecture



Figure 1. Verified telemetry path from the simulator to MongoDB Atlas.

Ride requests follow a parallel event path: the authenticated REST API sends a request to SQS, the matching worker atomically claims a nearby idle vehicle, AWS IoT Core sends the dispatch command, and the trip worker stores the completion event. SQS decouples the services and each queue has a dead-letter queue for repeated failures.

Implementation

Vehicle telemetry and monitoring

- A Node.js simulator models idle, to-pickup and occupied states and publishes position, speed, battery, status and timestamp fields using MQTT QoS 1.
- AWS IoT Core authenticates the vehicle with an X.509 certificate. An IoT rule forwards telemetry to the telemetry-ingest SQS queue.
- The telemetry service validates bounds and timestamps, rejects impossible GPS jumps, suppresses unchanged parked readings and calculates an H3 cell before writing to Atlas.
- Node-RED subscribes to fleet/vehicles+/telemetry and separates valid and invalid messages in its Debug sidebar. It is monitoring-only, so the ECS worker remains the single database writer.

Ride matching and trip lifecycle

- The API creates ride requests and returns a request ID. Every /api route requires a token stored in AWS Secrets Manager; /health remains public for load-balancer checks.
- The matcher reads SQS messages in short batches, searches nearby idle vehicles and uses a conditional MongoDB update to prevent two requests from claiming the same vehicle.
- The selected vehicle receives a command through the AWS IoT data plane. The simulator moves through pickup and drop-off and publishes a completion event.
- The trip service upserts by trip ID so repeated completion delivery does not create duplicate trips. The ride record contains ETA fields and requested, assigned and completed history entries.

Deployment and data design

- Docker images are stored in Amazon ECR and deployed to ECS Fargate. The latest verified revisions were API 3, matcher 2 and trips 2; every service reported one desired and one running task.
- The API runs behind an internet-facing Application Load Balancer. The task security group accepts port 3000 only from the load balancer security group.
- MongoDB Atlas separates current state from history. vehicles_live has geospatial and status indexes; position_history uses a seven-day TTL index; rides and trips use unique request or trip identifiers.

Progress against the nine week plan

Week	Planned outcome	Final result
1	Requirements and design	Completed
2	Vehicle and ride simulators	Completed at prototype scale
3	AWS IoT and MongoDB	Completed and verified
4	Telemetry processing	Completed with ECS persistence and Node-RED monitoring
5	Ride matching	Completed with conditional vehicle claim
6	Trip lifecycle	Completed with ETA and idempotent trip storage
7	Cloud deployment	Completed manually in Learner Lab
8	Load and correctness tests	Completed at controlled prototype scale
9	Report and demonstration	Final report completed

Evidence of implementation

The following original Atlas captures show that the cloud telemetry path updated both the current vehicle record and the historical collection. The records include the vehicle identifier, status, timestamp and H3 spatial cell.

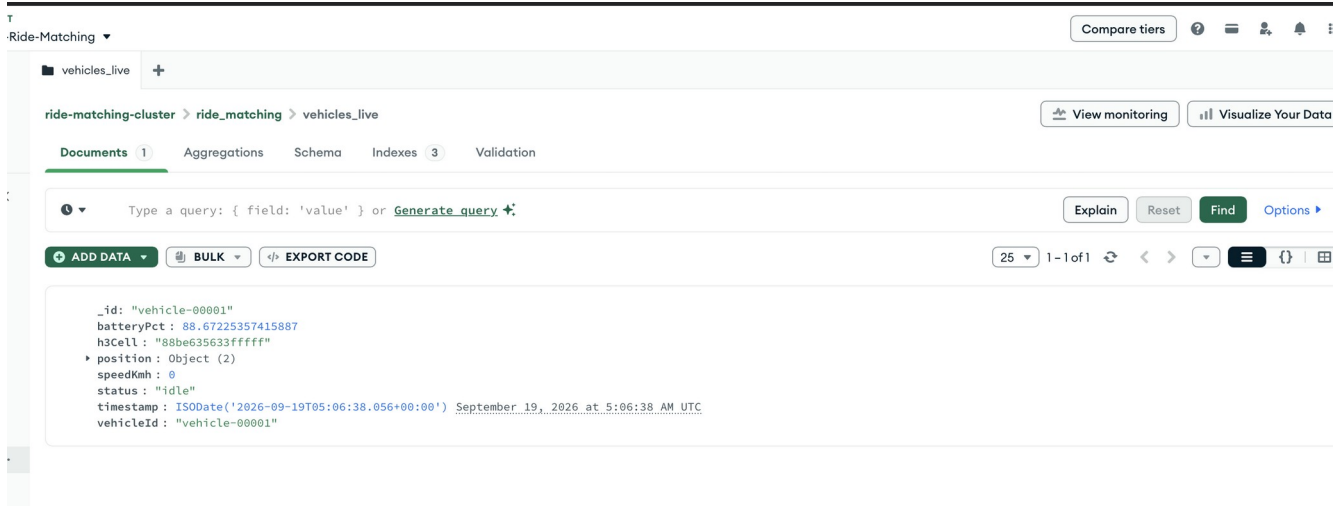


Figure 2. MongoDB Atlas vehicles live collection containing vehicle-00001.

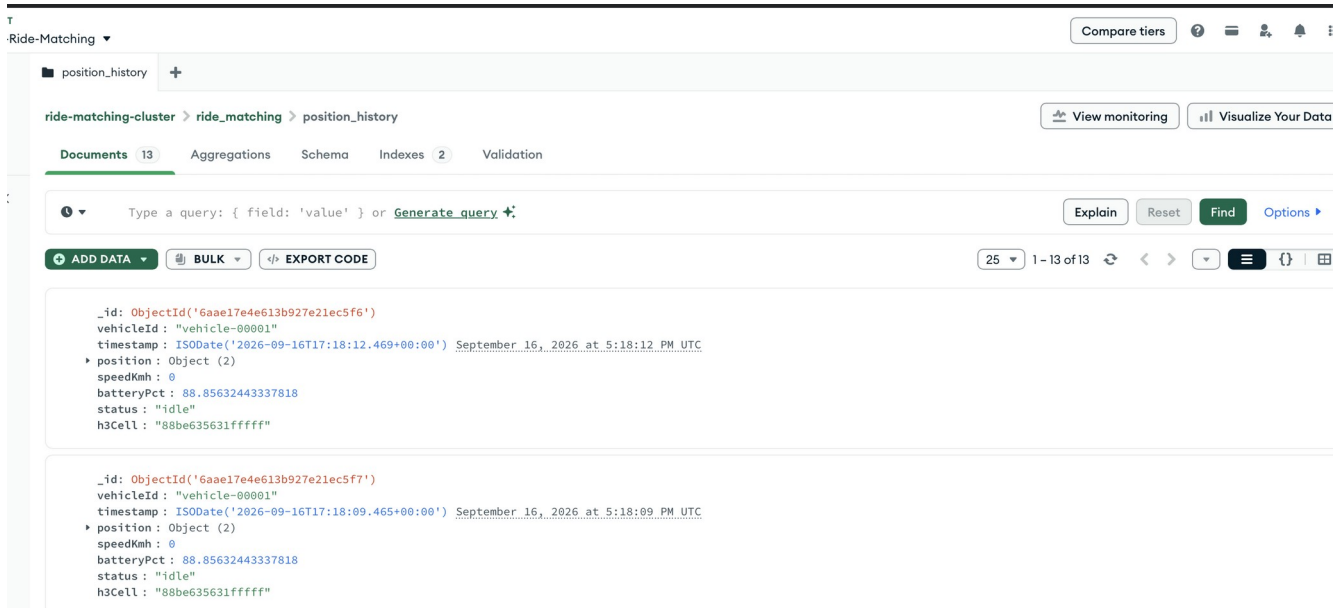


Figure 3. MongoDB Atlas position history containing multiple telemetry documents.

Functional verification

- The simulator logged a successful MQTT/TLS broker connection and published telemetry to the vehicle topic.
- The end-to-end verifier published a fresh sample and found the same timestamp in Atlas, proving the IoT Core, IoT rule, SQS, ECS and Atlas path for that message.
- An authenticated ride request completed with vehicle-00001. The stored history contained requested, assigned and completed transitions, and the trip record contained distance, duration and fare.
- Node-RED displayed both controlled branches: valid telemetry produced no validation errors, while an invalid sample reported topic ID, coordinate, speed, status and timestamp errors.

Testing and scalability results

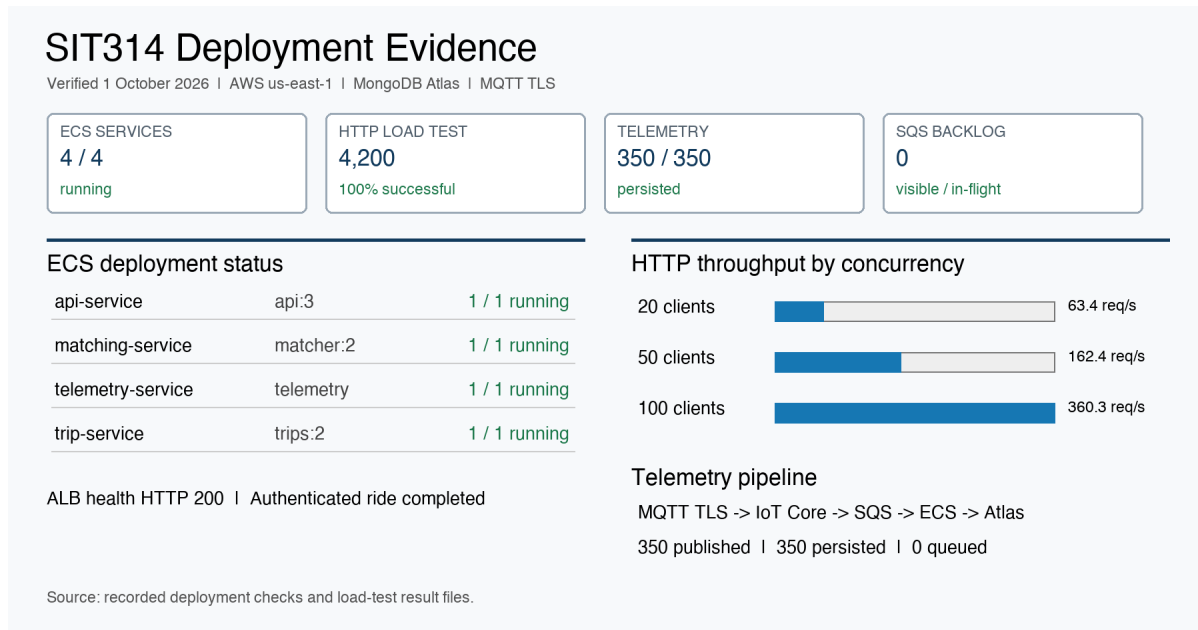


Figure 4. Consolidated deployment, HTTP load and telemetry verification results.

HTTP load test

- 200 requests at concurrency 20: 63.40 requests per second, p95 513.92 ms and 100 percent success.
- 1,000 requests at concurrency 50: 162.40 requests per second, p95 531.48 ms and 100 percent success.
- 3,000 requests at concurrency 100: 360.33 requests per second, p95 474.09 ms and 100 percent success.
- Maximum API CPU utilisation was 7.071 percent. One task remained sufficient because the 60 percent scaling target was not reached.

Telemetry rate test

- The three stages published 350 messages in total. All 350 timestamps were found in position_history.
- Publish throughput was 3.74 to 3.80 messages per second. Maximum telemetry service CPU utilisation was 7.609 percent.
- After both experiments, telemetry-ingest, ride-requests and trip-events reported zero visible and zero in-flight messages.

Evaluation and conclusion

Correctness tests

The final local run passed 15 of 15 tests. Coverage includes authentication, geographic calculations, H3 stability, telemetry validation, one-to-one assignment, maximum match distance, ETA calculation and stable trip-completion retries.

What the results demonstrate

- The prototype implements the complete telemetry and ride lifecycle using managed cloud messaging and independently deployed services.
- SQS isolates producers from consumers, while idempotent trip upserts and conditional vehicle claims protect two important correctness boundaries.
- The HTTP service remained available throughout the controlled test, and the telemetry pipeline persisted every measured sample without a remaining queue backlog.

Limitations

- The telemetry experiment used one authenticated Thing certificate. It measures message rate for one vehicle and does not prove the proposed 500-to-5,000 vehicle identity scale.
- The API load test targeted /health. It measures the public request path but not MongoDB ride creation, matching latency or dispatch latency under equivalent load.
- The single ECS task never approached the CPU scaling threshold, so automatic scale-out was configured but not triggered during the recorded experiment.
- Deployment was manual in AWS Academy Learner Lab. Infrastructure as code, private Atlas connectivity, multi-zone matcher ownership, distributed tracing and failure-injection tests remain future work.

Conclusion

The project meets the final task requirement by delivering a working and measurable IoT solution. The evidence proves the end-to-end prototype at controlled scale: secure MQTT telemetry reached Atlas, authenticated ride requests completed through the event pipeline, all automated tests passed, and the HTTP test sustained increasing concurrency without failures. The report does not claim production readiness or a verified 5,000-vehicle deployment because those conditions were not tested.

Appendix

Project code and configuration submitted with this report include:

- Services: src/services/api.js, telemetry-service.js, matching-service.js and trip-service.js.
- Simulation and load tools: vehicle-simulator.js, ride-generator.js, load-profile.js, http-load.js and telemetry-load.js.
- Configuration: Dockerfile, docker-compose.yml, infra/mosquitto.conf, node-red/flows.json and scripts/deploy-ecs.sh.
- Evidence data: docs/evidence-2026-09-30.md, scaling-results-2026-10-01.json and telemetry-scaling-results-2026-10-01.json.
- Security note: certificates, private keys, temporary AWS credentials, API tokens and the MongoDB connection string are excluded from the repository and report.

Submission checklist

- Submit project_report.pdf and the project source/configuration archive to OnTrack.
- Discuss the report and evidence with the tutor for sign-off.